RE Report - Verification

Professor: Subir Kumar Roy

Name: Nupur Dhananjay Patil

Roll number: IMT2022520

Email: Nupur.Patil@iiitb.ac.in

Date: 3rd May 2026

Github repo: https://github.com/NupurP04/RE_verification

1. Getting familiar with SystemVerilog

I began by reading the book SystemVerilog for Verification. At the same time, I studied the slides you shared on “Functional Verification Overview”. These slides explained why verification is important, drawbacks of simulation-based verification, and different verification methods like simulation, coverage-driven verification, assertion-based verification, formal verification, emulation, and prototyping. The slides also showed in detail how a typical UVM-style testbench is structured, including components like test, generator, agent, driver, monitor, scoreboard, checker, and DUT.

2. Writing the Verification Plan for MIPS Processor

After learning the basics, I tried preparing a verification plan for a MIPS processor. I first understood the 5-stage pipeline (Fetch, Decode, Execute, Memory, Write Back) and the different types of instructions it supports. I listed out possible errors that can occur in the design and what my verification code needs to catch. As per your guidance, I focused mainly on data hazards (RAW, WAR, WAW). I also studied interrupts, exceptions, and sub-routines in MIPS, including how the cause register and status register are used for handling them. This plan helped me decide what features and corner cases the testbench should cover.

3. Study of Assertion-Based Verification

Parallel to the above, I studied assertion-based verification using your slides. I learned how to write assertions directly in the code to check protocol rules and design behavior, and how they can be used both in simulation and formal verification.

4. In-depth Session on Formal Verification

You took a complete detailed session on Formal Verification. In this session I learned different paths of computation and how we prove properties on them. Linear Temporal Logic (LTL), we pick one path from the computation tree and prove the property holds on that path and then it has to be proven true in all other paths. Computation Tree Logic (CTL), prove property on subtrees, we have the option of choosing a single path/subtree/path in a subtree and only 1 path needs to be validated. It uses path modalities A, E. Then about classification of properties: Safety properties where undesirable states are never reached or desirable things always happen and Liveness properties where desirable states are repeatedly/ eventually reached. We also discussed about vacuous truth and absolute truth, and how different temporal modalities (G, F, X, U) work and can be converted into equivalent forms.

5. Detailed Study on Data Hazards

You asked me to focus specifically on RAW, WAR, and WAW hazards. I studied how these hazards are caused in a pipelined processor and how they can be prevented (using stalling or forwarding). I prepared a separate document explaining all three hazards with examples and prevention methods. I have attached this document in the github link for your reference.

6. Building the Complete Testbench

After the plan was ready, I started writing the actual verification code in SV using a UVM-style structure with mailboxes for communication between components and events for triggering and launching events. I created the following files:

- mips_interface.sv
- mips_transaction.sv
- mips_driver.sv
- mips_monitor.sv
- mips_scoreboard.sv
- mips_environment.sv
- mips_generator.sv
- mips_tb_top.sv

Please refer to the github link for these files.

I also used a behavioral 5-stage MIPS processor code (mips_processor.sv) that I found online as the DUT and I made a few changes to that.

7. Challenges in the Generator Class and Final Test Cases

I faced some difficulties while writing the generator class. I had already implemented mailboxes, a software register file (to keep track of expected values independently of the DUT), logic to calculate expected results, and RAW hazard detection. I was stuck on how to properly write the test sequences. Later I completed the generator and created a total of 13 instructions with cases of 0-gap RAW (expected 2 stalls), cases of 1-gap RAW (expected 1 stall) and cases of 2-gap RAW (expected 0 stalls)

8. Results

After connecting all components and making small fixes in the environment and scoreboard (moving generator before fork and changing ‘join_none’ to ‘join’), I ran the full simulation. The scoreboard gave the following final summary:

Total checked : 13
Value PASS : 2
Value FAIL : 11
Stall PASS : 7
Stall FAIL : 6

The scoreboard compares expected vs observed in FIFO order, and the monitor correctly samples writeback signals. The testbench correctly generates RAW hazard scenarios, computes expected results, and compares them using a scoreboard. The observed mismatches are due to incorrect DUT behavior. The DUT fails to insert required stalls for RAW hazards and also shows pipeline inconsistencies where incorrect destination registers are written.

The testbench worked correctly and successfully caught the bugs in the processor's hazard detection and stalling logic.

Conclusion

This semester gave me a strong understanding of both the theory and practical side of functional verification. I learned SV testbench development, coverage-driven verification concepts, assertion-based verification, and the basics of formal verification. Building the complete MIPS testbench helped me apply everything I studied in a real design.

Thank you for all the guidance and detailed explanations throughout the semester.